

HealthRadar

Disease Propagation Simulator

P.G.L. Cellules 2D – ING1 GI1 – CY Tech 2025-2026
Java 21 + JavaFX 11 | MVC Architecture | SEIRD Model

Authors: ABAL Marc-Antoine, BEN GHORBEL Khayem, TRAN TU THIEN Thémis,
COLLIN Gweltaz, SAAD Mohamed, GHOMMAM Mahdi

Teachers: Eva ANSERMIN – Romuald GRIGNON

1. Application Overview

HealthRadar is a 2D cellular automaton that simulates the propagation of infectious diseases across an urban grid. Each cell in the grid represents one person. The simulation follows an **SEIRD** epidemiological model: Susceptible → Exposed → Infected → Recovered → (back to Susceptible), with a probability of dying at the end of the infectious period.

The application is built with **Java 21** and **JavaFX 11**, structured following the **Model-View-Controller (MVC)** pattern, and is compiled and launched via a **Makefile**.

2. Quick Start

Requirements

- Java 21 JDK (javac, java, jar, javadoc)
- OpenJFX 11 installed at `/usr/share/openjfx/lib/` (Ubuntu: `apt install openjfx`)
- GNU Make

Available Make targets

Target	Action
make	Compile + package HealthRadar.jar (default)
make compile	Compile .java sources to out/
make jar	Compile + build HealthRadar.jar
make run	Compile, package, and launch the application
make doc	Generate JavaDoc in docs/
make clean	Delete out/ and HealthRadar.jar
make distclean	Delete everything including docs/

First run

```
git clone <your-repo> healthradar && cd healthradar
```

```
make run
```

On the presentation day you must **not** use an IDE: clone, compile, and run entirely from the terminal with the Makefile targets above.

3. Disease Model (SEIRD)

Each person on the grid transitions through the following states:

State	Colour	Description
EMPTY	Light grey	No person occupies this cell.
SUSCEPTIBLE	Blue	Healthy; can be infected.
EXPOSED	Orange	Incubating; not yet contagious (duration: incubationPeriod steps).
INFECTED	Red	Contagious; spreads disease to neighbours.

RECOVERED	Green	Immune for immunityDuration steps, then returns to Susceptible.
DEAD	Dark grey	Died from the disease; no longer active.

Transmission modes

Contact mode

An INFECTED cell can only transmit to cells in its immediate neighbourhood (radius = 1, up to 8 neighbours). This models diseases spread by physical contact or short-range droplets (e.g. Influenza).

Airborne mode

An INFECTED cell transmits to all SUSCEPTIBLE cells within a configurable Euclidean radius (default 3). This models airborne pathogens (e.g. COVID-like).

Probability model

The effective infection probability for each susceptible target is:

$$P(\text{infection}) = \text{disease.transmissionRate} \times (1 - \text{cell.resistance})$$

Each cell has a personal *resistance* drawn from a normal distribution (mean 0.2, σ 0.1, clamped to [0, 0.6]). This models individual immune differences.

4. Preset Diseases

Parameter	Influenza	COVID-Like
Mode	Contact	Airborne (r=3)
Transmission rate	30%	20%
Incubation	3 steps	5 steps
Infection duration	7 steps	14 steps
Mortality rate	1%	2%
Immunity duration	30 steps	60 steps

A **Custom** mode lets you override every parameter with the sliders in the right sidebar.

5. The Grid

Dimensions

Default: **60 columns × 45 rows** (configurable in code or via future settings). Each cell is rendered as a 14×14 pixel square on the canvas.

Topology

- **Bounded** (default): cells at the border have fewer neighbours; the simulation 'ends' at the edges.
- **Toroidal**: the left edge connects to the right, top to bottom. Toggle with the checkbox in the sidebar.

Step mechanics (per simulation tick)

- **Phase 1 – Movement**: each alive cell attempts to move to a random adjacent empty cell with its personal moveProbability (0.1–0.4 per step). Cells are processed in random order to avoid directional bias.

- **Phase 2 – Infection:** each INFECTED cell tries to transmit to SUSCEPTIBLE cells within its range. A double-buffer snapshot ensures all cells read the previous generation.
- **Phase 3 – Progression:** state ages are incremented; EXPOSED cells become INFECTED after incubationPeriod steps; INFECTED cells either die (with mortalityRate) or recover; RECOVERED cells become SUSCEPTIBLE again after immunityDuration steps.

6. User Interface

Top toolbar

Control	Action
■ Play	Start the automatic simulation loop.
■ Pause	Pause at the current step.
■ Step	Advance exactly one step (pauses first).
■ Reset	Clear the grid and reset the step counter.
Speed slider	Sets steps per second (1–20).
Disease	Select Influenza, COVID-Like, or Custom preset.
Mode	Switch between Brush / Zone / Individual edit modes.
Paint	Choose the state painted by mouse actions.
■ Save	Serialise the full simulation to a .hrs binary file.
■ Load	Restore a previously saved .hrs file.

Edit modes

Brush mode

Click or drag on the canvas to paint cells one by one with the selected state.

Zone mode

Click and drag to draw a rectangle; all cells inside are filled with the selected state when you release the mouse. A yellow outline shows the selection.

Individual mode

Single click on one cell to change it to the selected paint state.

Right sidebar

Disease Parameters panel

Four sliders let you override the active disease parameters in real time:

- Transmission rate (0.01–1.0)
- Mortality rate (0–0.5)
- Airborne radius (1–8 cells)
- Airborne checkbox: toggles contact vs. airborne mode

Random Populate

- Susceptible %: fraction of cells to fill with SUSCEPTIBLE people.
- Infected %: fraction to fill with INFECTED people.

- ■ Random Populate: clears the grid and fills it with these ratios.
- ■ Clear Grid: empties all cells.

Toroidal topology checkbox

When checked, the grid wraps around at all four edges.

7. Live Statistics

The stats panel (top of the right sidebar) updates after every step and shows:

- **Step counter** and total population.
- **Per-state counts and percentages** (Susceptible, Exposed, Infected, Recovered, Dead).
- **Stacked percentage bar** showing the current population split.
- **Time-series line chart** with one coloured line per state, covering up to the last 500 steps.
- **Colour legend** matching the grid and chart colours.

8. Save / Load

At any moment you can save the complete simulation state to disk:

- Press ■ **Save** and choose a file name (extension *.hrs*).
- The entire engine (grid, cell states, disease parameters, step history) is written as a binary Java-serialised object.
- Press ■ **Load** to restore any previously saved file and resume exactly where you left off.

9. Code Architecture (MVC)

Package	Class	Role
healthradar	App	JavaFX entry point; launches MainController.
healthradar.model	CellState	Enum: EMPTY, SUSCEPTIBLE, EXPOSED, INFECTED, RECOVERED, DEAD.
healthradar.model	Cell	One person: state, stateAge, resistance, moveProbability.
healthradar.model	Disease	All epidemiological parameters; two factory presets.
healthradar.model	Grid	2-D array of Cells; runs the 3-phase step loop.
healthradar.model	SimulationEngine	Owns the Grid; drives stepping; records StepStats history.
healthradar.view	GridView	JavaFX Canvas; renders cells, hover, zone selection.
healthradar.view	StatsPanel	JavaFX VBox; summary text, line chart, legend.
healthradar.view	EditMode	Enum: BRUSH, ZONE, INDIVIDUAL.
healthradar.controller	MainController	Builds the scene; wires all events; drives AnimationTimer.
healthradar.io	SimulationSerializer	Binary save/load via Java ObjectOutputStream.

The double-buffer pattern in *Grid.step()* deep-copies the cell array before each tick, so the infection and movement phases always read the *previous* generation and write to a fresh copy, avoiding order-dependent artefacts.

10. Project File Structure

```
healthradar/
  ■■■ Makefile
  ■■■ MANIFEST.MF
  ■■■ HealthRadar.jar ← built by 'make jar'
  ■■■ docs/ ← JavaDoc (built by 'make doc')
  ■■■ src/main/java/healthradar/
  ■■■ App.java
  ■■■ controller/
  ■ ■■■ MainController.java
  ■■■ model/
  ■ ■■■ Cell.java
  ■ ■■■ CellState.java
  ■ ■■■ Disease.java
  ■ ■■■ Grid.java
  ■ ■■■ SimulationEngine.java
  ■■■ view/
  ■ ■■■ EditMode.java
  ■ ■■■ GridView.java
  ■ ■■■ StatsPanel.java
  ■■■ io/
  ■■■ SimulationSerializer.java
```

11. Ethical Considerations

HealthRadar was designed with the following ethical principles:

- **Fairness:** the simulation treats all grid cells equally; no zone is structurally advantaged.
- **Transparency:** all parameters are visible and adjustable in the UI; no hidden mechanics.
- **Privacy:** the application does not collect, transmit, or store any real personal health data. Simulated data remains local.
- **Responsibility:** the simulation is a simplified model for educational purposes only and must not be used to make real public-health decisions.
- **Accessibility:** the interface is usable with keyboard shortcuts and the colour palette has been chosen for basic colour-blind readability.